

反作弊统一架构 总体设计

模块名称

所属系统 大禹

模块负责人 周杰

项目负责人 郭乐，杨佳学，周超，周杰

文档提交日期 2015.03.09

@version: v0.1 设计方案初稿

@version: v0.2 分级和分层过滤描述，完善流式系统设计（容错、顺序消费等），组件设计独立章节

@version: v0.3 完善数据层组件设计，调整架构增加TaskMerger

@version: v0.4 完善各worker描述

@version: v0.5 按照总体设计重新组织章节，增加单独容错方案描述、统一IO接口等

目录

1. 背景
2. 名词解释
3. 设计目标
 - 3.1 实现的功能
 - 3.2 设计的性能指标
4. 设计思路及折衷
 - 4.1 多级过滤和过滤逻辑分层
 - 4.2 数据层组件
 - 4.3 特征层组件
 - 4.4 过滤层组件
 - 4.5 水平扩展
 - 4.6 容错方案
 - 4.7 有序消费
 - 4.8 统一读写接口
5. 系统设计
 - 5.1 整体方案描述
 - 5.2 XExtractor
 - 5.3 XAccumulator
 - 5.4 TaskMerger
 - 5.5 XFilter
 - 5.6 LogMerger

1 背景

反作弊是百度广告日志流式处理流水线中重要的一个环节。在过去的两年里，我们基于统一的流式计算框架（Task Manager）完成大禹项目一二期，分别重构了原有广告检索日志批量过滤系统（Remora）和广告点击单机过滤系统（Dorado），在提升过滤实效性的同时改善了系统的可维

护性。在大禹一二期中为了控制项目风险，我们在重构的过程中最大程度地复用了原有检索、点击过滤系统的策略设计与实现，保证了大禹系统切换上线时策略效果同原有系统的效果差异最小化。但是原有策略的固有缺陷依然存在，并随着接入反作弊的产品线和产品线流量的增长而愈发显著。主要问题有以下几点：

1. 点击系统中策略的串行处理模式。即一条日志一旦被某策略过滤，其对此策略之后的所有策略都不可见，这种单机处理模式是凤巢发展初期的产物，策略无法并行处理。但是在百度商业产品日新月异的今天，它显然已无法承载流量的快速增长。大禹二期点击系统重建于流式分布式框架之上，但是却没有真正享受到分布式的优势，核心过滤模块仍然是单点模块，吞吐不足或内存超过单机限制时，当前只能串行部署多个过滤模块（凤巢已经拆成4个过滤模块），维护成本高，无异于饮鸩止渴。在统一规划下，策略RD在14年Q4通过繁琐的调研和调参完成了点击系统的策略间去依赖，这为点击系统并行化实现提供了可行性。
2. 两套系统模块、策略异构。本质上，点击系统的策略和展现系统策略虽然处理逻辑上是一致，但是长期以来两套系统是完全独立的两套实现。即便是相同的功能Feature，从开发、测试到运维以及周边系统支持都需要双倍的人力成本。在前一个问题的介绍中提到，点击并行化架构的前置依赖都已消除，那么检索点击的策略、框架同构也便成为可能。
3. 过滤框架无法满足策略发展的新需求。策略对流式过滤框架的新需求主要体现在两方面。一方面为流式框架需要支持多特征的联合过滤。当前两种系统均为单特征一票否决的过滤模式，在这种模式下我们很难进一步提高复杂作弊的召回率。在引入多特征共同过滤的基础上我们也就能方便地加入对模型过滤的支持。另一方面在过滤时需要引入更多参考特征，例如点击、检索特征的相互参考和过滤时参考转化数据等。

本项目通过统一反作弊系统架构，力求解决上述问题。长远来看统一架构将极大简化反作弊系统的设计，降低维护成本，提升系统迭代效率。项目一期主要考虑在架构上同时支持点击和检索系统，策略上主要完成对点击策略的支持。

2 名词解释

3 设计目标

3.1 实现的功能

统一架构项目一期的主要功能点包括：

1. 点击检索策略组件同构。完成统一的策略组件抽象和接口设计，同时支持点击、检索系统的流式、离线策略，完成点击系统当前支持的所有策略的实现。
2. 点击检索模块同构。完成统一的点击检索过滤系统架构设计，实现流式和批量框架版本。
3. 支持多特征过滤。实现兼容现有策略的多特征规则过滤策略，抽象模型过滤接口。

3.2 设计的性能指标

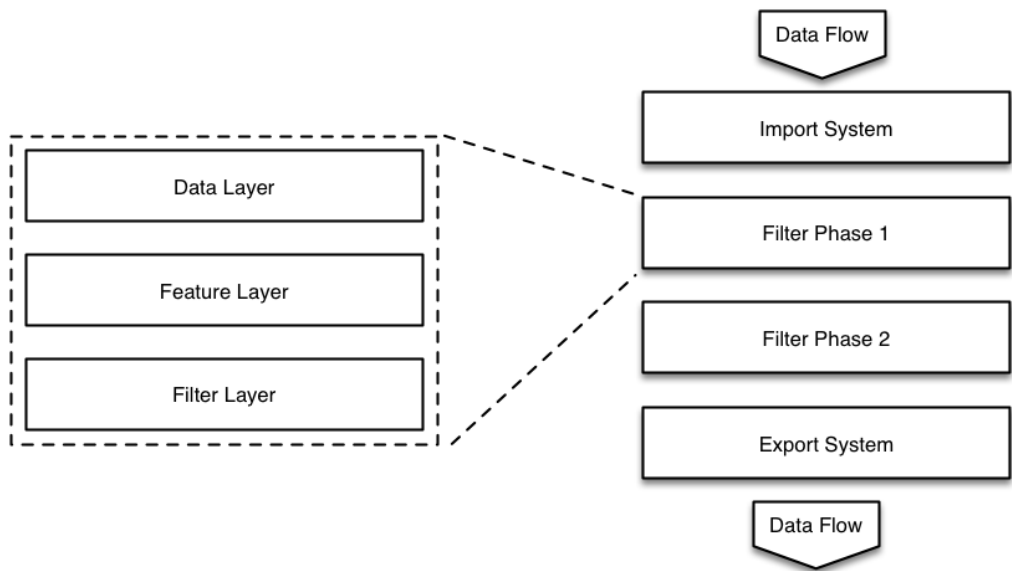
4 设计思路及折衷

4.1 多级过滤和过滤逻辑分层

广告点击流量中，简单作弊形式（爬虫、刷词等）波动幅度大，但是容易识别，预先过滤掉简单作弊形式可以使剩余流量之上统计特征的表现更加稳定，防止流量较小的高级作弊形式淹没在大量的简单作弊中。因此点击过滤系统dorado设计之初就采用了策略串行的过滤方式，即日志一旦被某策略过滤则对其后的策略不可见。相对于并行策略，串行策略不易于水平扩展。因此我们采用折衷的多级过滤方案。

多级过滤的方案是层级内的策略并行过滤，过滤层级之间串行过滤。一旦日志被某过滤层级的若干策略过滤，框架将为这些已过滤日志打上标记透传给下游，但是它们对之后的过滤层级策略不可见。这样能在一定程度的串行过滤效果上实现系统的水平扩展。此外，为了保证过滤层级的可叠加，过滤级的输入输出均为原始日志。

在每个过滤阶段内，逻辑上可以分为数据层、特征层、过滤层。分层主要是为了易于扩展，使层内的扩展对其他层透明。大禹现有的过滤系统都只有数据和过滤两个的逻辑层次，数据层输入日志，输出维度。过滤层输入维度、输出过滤结果。特征是在过滤层内部计算的。在这种分层设计中要实现多特征过滤策略会存在冗余的特征计算。例如策略p1\p2都需要计算某特征f1，两层结构中p1和p2必须分别计算f1。将特征层和过滤层拆分后特征层输入维度，输出特征值。过滤层输入特征值，输出判定结果。三层抽象之上能够较容易地支持多特征策略（包括模型过滤），此外单独的特征层也使外部特征值的引入以及内部特征值的导出成为可能。多级过滤和过滤逻辑分层的关系如下图所示。



4.2 数据层组件

数据层需要支持多元化日志的解析和访问, 对其他层提供统一的接口。沿用大禹中的名词定义如下:

- **LogRecord**: 日志行的抽象, 提供对维度和维度组合的访问。
- **维度 (View)** 和 **维度值 (ViewValue)**: 维度即用于特征计算的日志字段或转换后的日志字段, 一个日志行对应多个维度, 每个维度对应一行配置。维度对象是某行特定日志中维度的实例, 例如某行日志ip维度的实例为10.0.0.1维度对象。

统一多日志的访问接口主要由于ASP日志的复杂性而存在如下问题

- ASP日志是复杂的多层级结构, remora的统计策略是在广告级别、src级别的维度上统计, 统计值异常则过滤整个PV。
- ASP日志的维度组合是将pv级别维度join到广告级别字段, 并非简单的笛卡尔积。例如ip,winfo组合, 某日志ip为10.0.0.1 winfo在两个数据源有三条广告1:10,11;204:13, 则维度组合有<10.0.0.1,10>,<10.0.0.1,11><10.0.0.1,13>三个实例
- ASP日志中care_src功能是针对广告来源级别的过滤, 此功能作用于维度组合之前。例如上例中care_src为204, 则只获取到<10.0.0.1,13>一个实例。
- ASP日志中view_reject类似于care_src可以过滤各级别的维度。

由于ASP日志的多级复杂结构及现有系统中存在针对ASP特化的组件设计。访问接口的统一存在较大争议。对LogRecord的抽象主要有以下几种方案

- a. LogRecord表示单条日志, 维度组合及过滤 (caresrc) 在LogRecord内部完成
 - 优点: 接口最简洁, 一条日志, 多个字段间没有关系。
 - 缺点: 维度膨胀, 有关联的维度需要在Record内部combine, 例如提供ip_winfo、ip_winfo_src160字段。
- b. LogRecord抽象为一条日志, 维度组合及过滤(caresrc)由外部驱动 (实现上可能是特征层调用LogRecord获取字段时传入回调过滤函数 或 获取维度时携带附属信息, 特征抽取拿到字段后筛选附属信息)
 - 优点: 接口相对简洁, 类似于现有remora的方式
 - 缺点: 数据层和特征层都需要针对ASP日志有独立的实现, 才能够识别不同回调函数或附属信息。
- c. LogRecord抽象为多条日志, 例如一行ASP日志会抽象为一行PV级别LogRecord, n个src级别LogRecord和m个广告级别LogRecord, 维度组合和过滤都在特征抽取时单条LogRecord通过CareSpace完成
 - 优点: ASP复杂的字段间关系在拆分LogRecord时消除了, CareSrc逻辑都可以转换为某个子Record的CareSpace实现。
 - 缺点: 理解和使用可能不够直观。策略需要明确指定获取LogRecord的类型。(补救措施: 在某日志只有一种LogRecord时可以省略特征配置的Record类型)

综合各优缺点, 达成一致采用c方案。代码描述如下

```
// 维度描述
```

```

class ViewDescriptor {
public:
    std::string name() const;
    RecordType record_type() const;
    int index() const;
};

// 维度值
// c++基本类型值或数组类型
class ViewValue {
public:
    enum ViewValueType {
        INT32,
        UINT32,
        INT64,
        UINT64,
        FLOAT,
        STRING,

        INT32_ARRAY,
        // ...
    };
    ViewValueType type() const;

    size_t size() const;
    uint64_t sign(int idx) const;

    // 明文访问接口
    int32_t int32_value() const;
    // ...
};

// 日志解析访问接口
// 使用方式示例如下
// LogRecordInterface* logrecord;
// logrecord->parse_from_string(line);
// while(logrecord->has_next()) {
//     logrecord->next();
//     for (e : extractors) {
//         e->extract(logrecord);
//     }
// }
//
// 在一个Extractor内部
// bool extract(const LogRecord& logrecord, FeatureValue* feature) {
//     ViewDescriptor* ip = logrecord->find_view_by_name("ip");
//     ViewDescriptor* winfo = logrecord->find_view_by_name("winfo");
//     vector<const ViewDescriptor*> view_desc = {ip, winfo};
//     ViewValue value;
//     logrecord->get_view_value(view_desc, &value);
//     process(value, feature);

```

```
// }
//
// 继承关系如下
//
//                                     <|-- AspLogRecord
// LogRecordInterface
//                                     <|-- FlatRecordBase  <|-- ClickPbRecord
//                                     <|-- XXXPbRecord
//
class LogRecordInterface {
public:
    virtual bool parse_from_string(const std::string& line) = 0;
    virtual void clear() = 0;
    virtual bool has_next() const = 0;
    virtual bool next() = 0;
    virtual enum record_type() const = 0;

    virtual const ViewDescriptor* find_view_by_name(const std::string& name) const = 0;
    virtual std::vector<const ViewDescriptor*> list_view() const = 0;

    virtual get_view_value(vector<const ViewDescriptor*> view, ViewValue* value) const = 0;
    virtual uint64_t record_id() const = 0;
    virtual uint64_t record_time() const = 0;
    virtual bool empty() const = 0;
}

```

维度配置为了便于使用，采用单行配置方式

```
# 配置包含四部分

# 维度名；维度所属LogRecord；维度提取函数(维度提取函数参数)；维度后处理函数(维度后处理函数参数)
# 说明：
# 1. 维度所属LogRecord可以不配置，只有在ASP日志会有多个Record时会用到
# 2. 维度提取函数描述为
#     accessor_function(in: parameters, in/out: this_view)
# 3. 维度后处理函数仅对当前提取的维度做简单的变换，可以描述为
#     post_process_function(in: parameters, in/out: this_view)
[AspLogRecord]
query          ASP_SEARCH_RECORD ViewAspDirect1($query)
               NULL
cntname        ASP_SRC_RECORD     ViewAspDirect2($cntname)
               NULL
wbwsid         ASP_AD_RECORD       ViewAspPVRReflection3("asp_log_field.xxx.wb
wsid") NULL
query_decode   ASP_SEARCH_RECORD ViewAspDirect1($query)
               Decode("utf-8")

```

4.3 特征层组件

特征层提供特征计算组件。它以输入维度，输出特征。

在本文上下文中，特征定义如下

- 特征（Feature）是从某个角度对当前日志的描述。包括基本特征和统计特征。基本特征是单条日志维度中能够抽取到的特征。统计特征是根据滑动窗口统计得到的特征。
- 特征值（Feature Value）是特征的实例。特征(值)签名（Feature Value Sign）将每一个特征值进行签名后的结果，（在忽略签名冲突时）和特征值一一对应。进行签名的主要目的是为了系统处理方便。特征签名可以简单表示为md5(feature_id, feature_view_value)。
- 滑动窗口。统计特征滑动窗口支持时间和PV坐标。大禹点击系统策略都采用PV窗口，其优势是在未知流量上表现稳定。

支持特征

- 单PV特征
 1. 处理空间特征
 2. 名单特征
 3. 维度值明文特征
- 统计（上下文）特征
 1. 数目
 2. 短窗口数目
 3. 比例
 4. 分布
 5. 频次分布
 6. 自动黑名单
 7. 时差特征
 8. 集中度分散度特征
 9. 全局数目

组件接口描述

```

message FeatureValue {
    // 公共字段

    required uint64 feature_id = 1;           // 特征id
    required uint64 view_sign = 2;           // 维度签名
    required FeatureType feature_type = 3;    // 特征类型
    required uint64 joinkey = 4;             // 特征与日志拼接key, 通常可唯一表示日志
    required uint64 shard_key = 5;           // 维度实例分发key, 通常情况下feature_id^view_sign
    required uint64 sort_key = 6;           // 维度实例排序key, 通常情况下等于绝对坐标

    // 特征类型特有字段
    // ...
};

```

统计特征的抽取累计逻辑分别封装在一个FeatureExtractor和FeatureAccumulator中，单PV特征只包含FeatureExtractor，接口描述如下。

```

class FeatureExtractor {
public:
    virtual bool init() = 0;
    virtual bool extract(const LogRecord& log_record, FeatureValue* feature) = 0;
};

class FeatureAccumulator {
public:
    virtual bool init() = 0;
    // Update.
    virtual bool update(const FeatureValue& input) = 0;
    // Update and query.
    virtual bool update(const FeatureValue& input, FeatureValue* output) = 0;
    // Query.
    virtual bool query(const FeatureValue& input, FeatureValue* output) const = 0;
};

```

特征配置在Extractor的ccumulator两个阶段都会使用。采用和Remora类似的配置方式，主要的变化是增加RECORD_TYPE和VERSION字段。前者是为了适应数据层接口的抽象。后者是为了实现简单的特征checkpoint兼容性检查。策略配置修改者自己保证修改的兼容性，VERSION只提供不兼容检查，即框架发现checkpoint和配置中VERSION不同则丢弃本特征的checkpoint。示例配置如下。

```

# CareSpace特征
[@FEATURE]
FEATURE_ID : 2

```

```
FEATURE_TYPE : FEATURE_CARE
RECORD_TYPE: CLICK_RECORD
VERSION: 1
CARE : ['cmatch':=225]
FEATURE_CARE: [*]
```

维度值明文

```
[@FEATURE]
FEATURE_ID : 9
FEATURE_TYPE : VIEW_VALUE
RECORD_TYPE: ASP_PV_RECORD
VERSION: 1
CARE : [*]
VIEW : ip,query
```

窗口数目特征

```
[@FEATURE]
FEATURE_ID : 3
FEATURE_TYPE : PV_SEG or TIME_SEG
RECORD_TYPE: CLICK_RECORD
VERSION: 1
CARE : ['cmatch':=225]
VIEW : ip,cntname
VERSION: 1
WINDOW_LENGTH: 10 (单位min)
STEP_NUM: 2
```

窗口内维度值累积值比例特征

```
[@FEATURE]
FEATURE_ID : 5
FEATURE_TYPE : PV_MULTI_RATIO or TIME_MULTI_RATIO
VERSION: 1
RECORD_TYPE: CLICK_RECORD
CARE : ['cmatch':=225]
VIEW : ip,cntname
VERSION: 1
WINDOW_LENGTH: 10 (单位min)
STEP_NUM: 2
FILTER: ['cmatch':=225]
REFER: ['cmatch'!=225]
```

窗口内主维度下副维度值分布特征

```
[@FEATURE]
FEATURE_ID : 6
FEATURE_TYPE : PV_DISTRIBUTION or TIME_DISTRIBUTION
VERSION: 1
RECORD_TYPE: CLICK_RECORD
CARE : ['cmatch':=225]
VIEW : ip
DATA_VIEW: cd_time
INTERVAL : 10,100,1000
```

```
VERSION: 1
WINDOW_LENGTH: 10 (单位min)
STEP_NUM: 2
```

4.4 过滤层组件

过滤层提供作弊判定策略抽象，策略的输入是特征集合，输出是判定结果（bool值表示）。规则策略和模型策略采用统一接口。策略分别过滤之后由Judge综合判定，一期Judge的预采用策略过滤结果加权求和的简单实现。

```
class Policy {
public:
    virtual bool init(const comcfg::ConfigUnit& config) = 0;
    virtual bool filter(const FeatureMap& feature_map, PolicyResult* result) const = 0;
};

message PolicyResult {
    uint64_t policy_id;
    float value;           // 策略结果连续值，传递到滤后日志
    bool is_spam;  // 用于
};

class Judge {
public:
    bool init(const comcfg::Configure& configure);
    bool judge(const vector<PolicyResult>& results, bool* is_spam) const;
};
```

示例配置如下

```

[@POLICY]
  policy_id: 1
  policy_type: LR_MODEL
  feature: 10011,10012,10013
  model_conf: "lr.conf"

[@POLICY]
  policy_id: 2
  policy_type: RULE
  # 规则判断策略, 每个rule结果是bool, rule之间是and关系
  [@RULE]
    type: SLIDING_WINDOW_NUM_LIMIT
    feature: 10012
    threshold: 50
  [@RULE]
    type: SLIDING_WINDOW_RATIO_LIMIT
    feature: 10013
    remain: 10
    threshold: 0.91

[@POLICY]
  policy_id: 3
  policy_type: RULE
  [@RULE]
    type: BLACK_LIST
    feature: 10014
    black_list: "black.list"

[@POLICY]
  policy_id: 4
  policy_type: CLICK_PROTECTION

# 综合判定
# 一期实现简单地采用策略判定结果加权求和的方式
[JUDGE]
  # 生效策略id:策略权值
  @POLICY: 1:1
  @POLICY: 2:0

```

4.5 水平扩展

基于可运维性考虑, 所有模块需要支持较方便地水平扩展。流量增长后能够增加实例实现系统容量的线性扩展。在资源使用不充分的情况下能够通过减少示例节省资源。在大禹系统中, 通常采用两种扩展方式。

- 无状态worker根据需求增加工作实例（worker实例和token数目）即可。TM-Worker的工作模式使得计算能够自然地均匀分摊在所有实例上。
- 有状态worker通过分shard扩展计算能力。所以当性能不足或资源使用不充分时只需要增加

实例，重新调整shard数目。在大禹流式处理的场景下，停流之后调整是能够接受的。

Shard模块扩容缩容方案

当前系统设计中，只有特征统计模块有大量的内存状态，分shard采用checkpoint持久化状态。每个shard都有独立的状态，所以要求在shard数目发生变更时能够较为简易地实现状态的迁移，简化运维代价。

几种实现方案：

1. 纯手动迁移，操作时停机得到全部的状态，然后通过离线脚本计算，按新的shard数目进行状态重建，得到新shard数下每个shard的状态。
2. 启动时计算，操作停机后，修改token中的shard数&增加/减少token数，实例启动后比较当前shard数和上次shard数，按照公式计算得到需要从哪几个shard状态中重建新shard数下该shard状态。
3. dump时计算，在接受到停机指令后，实例根据下次启动时的shard数进行状态划分，下次启动后，实例根据读取当前shard下的所有状态数据即可完成状态重建，其中有以下关键问题：
 - 确定下次启动的shard数，可以有两个方案，一是通过计算同类型的token数目来确定（不能缩容），二是通过读取shard数目配置。
 - shard下多个状态数据保存，实例停机时dump状态对策略的状态文件的命名需要设计，类似policy20001-11-2，表示20001策略从shard 11迁移到shard 1的状态数据
 - 多副本的状态迁移，可以约定0号副本下的shard 0 - n 之间进行迁移，1号下的shard 0 - n进行迁移，等等。

在实际运维中，扩容&缩容都是2倍或者1/2，可以考虑方案2。方案2在实现时只支持2倍扩容或者缩容，可以简化状态重建公式。

Token

所有worker都采用持有token再工作的方式，一旦抢到token就进入工作状态消费task，消费失败放弃token计算自动迁移到其他相同实例。一方面可以方便控制task并发消费程度。另一方面在单机环境异常情况下，运行实例可以主动放弃工作，避免无谓的task争抢。

4.6 容错方案

模块异常

在大禹系统中，存在三类模块，对应的容错方案如下：

1. 无状态

无状态Worker容错方案相对简单。采用同质、多实例部署。少量worker失败后不影响系统，worker可以快速恢复重启。如上节所述无状态模块也采用token确定工作实例，避免单机环境异常情况下，实例持有Task单始终消费不成功的状况出现。

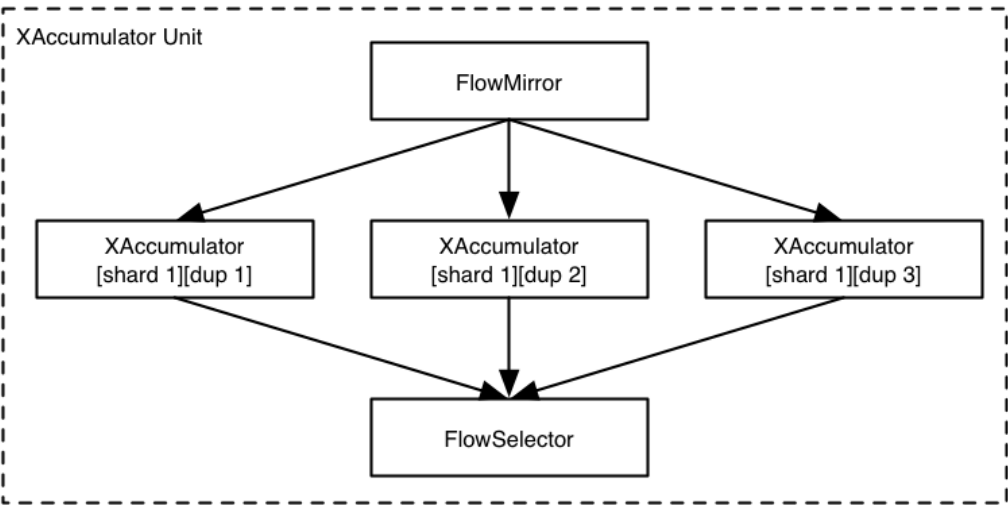
2. 有状态 – 容忍少量状态丢失

当前的检索日志过滤系统统计过滤模块采用hdfs的checkpoint持久化状态，能够容忍少量shard的checkpoint丢失。一方面由于shard数目比较多，单shard状态异常，对整体过滤效果的影响不大。另一方面检索数据量非常大，多副本代价较高。

1. 实例异常，状态不丢失。当出现任何异常但工作实例仍能正常导出最新checkpoint的情况下（例如网络抖动、TM无法访问等），实例会主动尝试释放token，计算通过token迁移到其他空闲实例。
2. 实例异常，状态丢失。对于单机死机、VFS故障等异常，内存状态无法持久化，累积状态丢失。

3. 有状态 – 可靠性要求高

点击日志流量小，shard数目少，状态的丢失可能导致过滤效果波动，进而造成公司收入波动。因此可以采用多实例热备的方案，多个实例处理镜像流量，处理顺序保证一致则内存状态在相同的Task处理之后完全相同，状态丢失的几率为多个热备实例同时丢失的概率。如下图所示，在每个shard的多副本热备模块上游增加流量镜像拆分模块FlowMirror，而在其下游增加选择模块FlowSelector选择生效流，删除备份流。



当前Selector的去重依赖上游Task的时序，而当前过滤模型中由于上游模块多实例消费引入了乱序，所以Selector的去重不能再依赖Task的时序，需要重新考虑其他去重方式，比如Task eventid的 hash table。但是这又会使得Selector变为有大量状态模块。另一种可行的方式是FlowMirror可以作为shard内全局协调者，为输入Task重新赋一个递增ID（例如DataTaskValue中增加字段），FlowSelector根据此ID的单调递增性质去重。需要保存的状态仅为last_id，可以放入其token中。拟采用后一种方案。

FlowMirror的功能可以直接集成在上游模块中，将其单独出来一方面是可以将热备功能独立，对上下游模块透明。另一方面也希望通过此模块保证多热备流的消费顺序严格一致，具体见下方有序消费章节。

某个热备实例的状态丢失后的恢复思路是从对应shard下进度快于它的其他热备流中拷贝可用checkpoint。自动化恢复方案如下：

token中增加运行状态 空闲 | 运行 | 恢复

1. 获取到token的实例可以据此状态判断之前token持有者是正常释放还是崩溃。正常释放为空闲状态，此实例将其改写为运行状态开始工作。
2. 如果发现token之前持有者异常退出，则清空对应checkpoint文件夹，在一个shard内worker共享的Group中产出一个dump checkpoint命令task。并将token切换到恢复状态。
3. 运行中的实例每轮结束后都尝试消费命令Task，消费时按照Task描述生成并拷贝checkpoint到指定位置。可以通过manifest文件等机制表示checkpoint完整。
4. 获取到恢复状态token的实例，则会检查checkpoint是否完整，完整则将其修改为空闲状态。

自动化的异常流恢复方案可以提高系统的可维护性。在具体实现中，还需要考虑以下细节：

- 备份流生成的checkpoint进度需要大于异常流当前的进度（checkpoint保存eventid对task进行去重）
- 性能上考虑，状态传递需要性能高效：
- 只用当前空闲的正常流（不被Selector选中过滤的那条流）进行传递。
- worker内部拷贝状态，异步dump状态。

全局异常

更极端的情况，HDFS集群故障、TM故障等全局，则通过机房热备的方式解决。

Task消费事务性

由于策略的统计操作非幂等操作，Task的消费需要保证事务性防止状态错误。即更新内存状态后出现异常导致消费失败（VFS unmount、TM restart），期望能将内存状态回滚到消费之前的状态。实现滑动窗口统计状态的回滚逻辑太复杂。remora采用了一种近似的方式，Filter提供累积过滤和只过滤两种接口，内存状态一旦更新，BundleId写入去重HashTable。一旦检测到重复Bundle，则直接调用Filter的过滤接口重新消费，不累积状态。采用这种方式获得的统计值为此Task消费前的状态快照，而非动态的累积过程中的统计值。因此只是一种近似的事务性模拟。

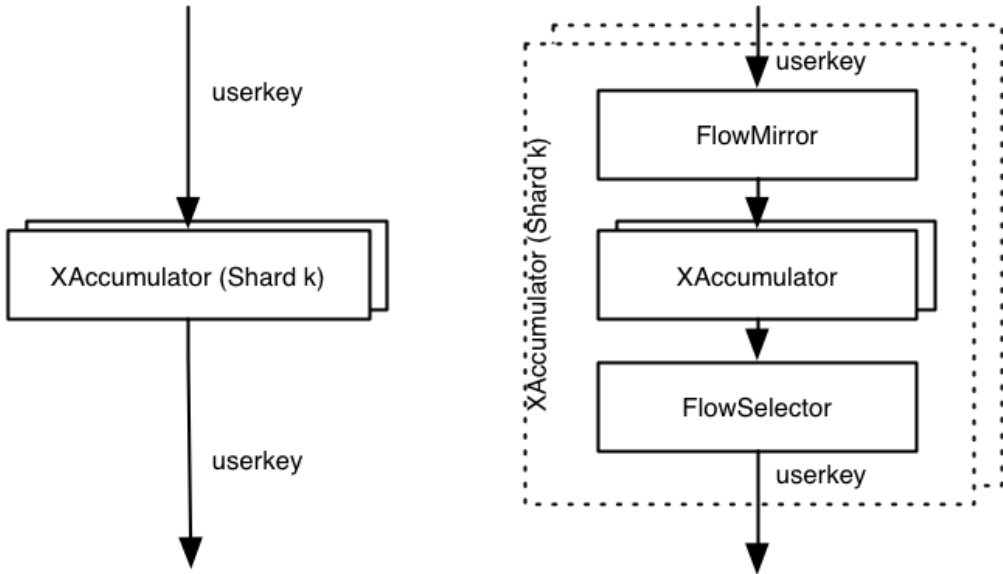
4.7 有序消费

统一架构之后，多实例并行消费可能会导致bundle进入系统入口的顺序和进入统计模块的顺序不一致。进而导致过滤效果不稳定。无状态模块不关心处理顺序。

从策略角度来看，日志不需要时间严格有序，小范围的乱序（滑动窗口步长范围内）对过滤结果基本无影响。检索策略只有少量的时差策略需要日志严格有序，当前采用缓存短时间的日志解决顺序依赖，检索系统通过对Task填充userkey = start_time+bundle_id（start_time用于排序，bundle_id用于拼接）消除上游bigpipe乱序的影响。大禹点击系统对于进入的数据流保证严格有序消费，是由于在系统设计时，下游计费系统依靠点击日志id的单调递增性来判重。计费系统在之后的迭代升级中去除了对日志id递增的约束，改用hashtable纪录已消费的点击日志。实际上上

游多机日志汇合到Bigpipe时已经没有严格时间序。所以可以统一采用检索的填充userkey排序Task的解决方案。

但是，在上述多流热备的方案下，如果采用userkey排序，多副本可能会由于消费速度不一致导致累积状态不一致，进而造成主备流切换时，过滤效果波动。因此可以在FlowMirror的输出中擦除Task中的userkey，按照输出Task的creation_time排序。FlowSelector在产出中重新填入userkey。除此以外全系统中Task都采用userkey=start_time+bundle_id。



无热备的Shard或者热备Shard的单个副本，都对应唯一的输入TaskGroup，消费模式为Query + TaskOwnership(Include owned)防止工作实例迁移后遗漏被上一个工作实例持有的Task。

4.8 统一读写接口

当前的TM框架下，没有提供统一的读写框架。导致每个worker都需要维护一套较为底层的读写逻辑来进行数据交互。这使得大量类似重复代码出现在不同的worker中，并且使得升级worker读取新的数据格式的成本异常高，不利用系统的发展。

为此在本项目中，我们考虑提供一套完整的读写接口来托管所有Worker在数据传输中所需要的文件操作，对应用层屏蔽底层数据层的工作方式，不仅可以减轻worker的开发代价，也能更好的扩展数据格式、数据传输方式。

接口上考虑对数据传输和数据格式进行分离，以reader为例：

```

class XReader {
public:
    bool init(enum file_type);
    bool open(XStream* stream);
    bool next(char* buf, int buf_size, int* o_size);
};

class XInputStream {
public:
    bool next(char** data, int* size);
};

class XFileInputStream: public XInputStream { };
class XNetInputStream: public XInputStream { };

```

数据格式

统一默认提供HDFS的数据传输方式，并且提供一种数据存储格式，该数据存储格式目标：

- 自描述，即无需借入Task中的描述信息也可以完成数据格式的读取、识别。
- 高可靠性，部分数据错乱情况下，不影响整个 Chunkfile/File 的读取。

设计的数据存储表示为：

```

File:
Chunkfile: [dayu_data_head_t] [DayuDataFileMessage] [dayu_data_head_t] [D
ayuDataFileMessage]
Chunkfile: [dayu_data_head_t] [DayuDataFileMessage] [dayu_data_head_t] [D
ayuDataFileMessage]

```

该存储下一个Chunkfile被拆分为多个小包，每个包都有一个固定size的head来描述，存储关于实际数据的meta信息。包与包之间没有关系，使得：

- 数据压缩从chunkfile转移到DayuDataFileMessage，降低了一次需要压缩的数据大小，减少了压缩所需buff大小
- 数据压缩从Task层，透明到文件中。解析文件时，可以通过数据内容来决定是否需要解压。
- 避免部分数据序列化失败导致全部数据解析失败，并且将部分数据进行聚合避免引入过多的附加信息
- 外围工具解析数据文件时，可以通过冗余附件信息，正确解析出包含的实际Message对象。

缺陷：

- 与现有的数据压缩方式相比，增加了压缩次数，降低了压缩比例

dayu_data_head_t的定义如下，主要包括magic_number和数据长度。

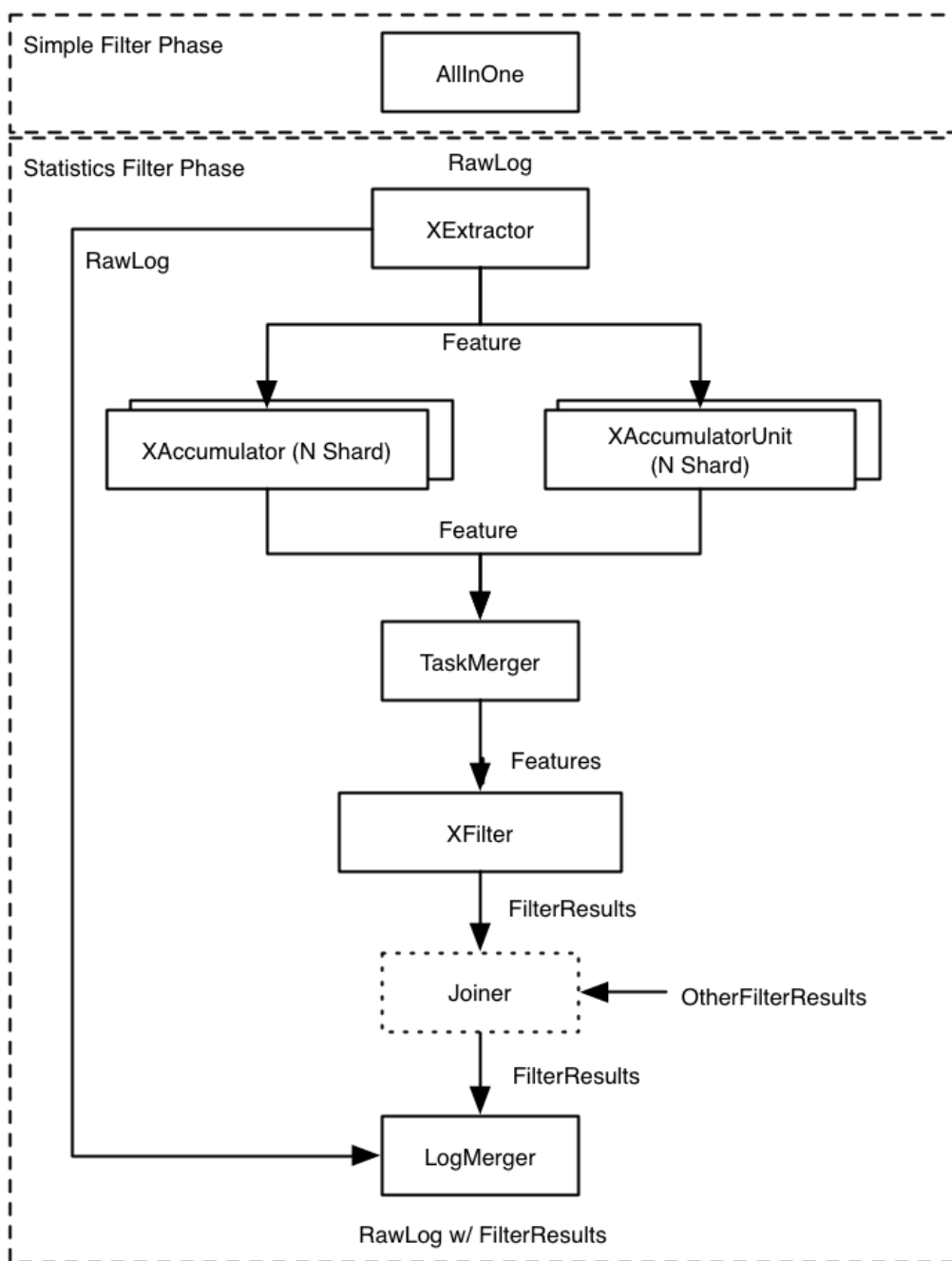
```
struct dayu_data_head_t {
    uint32_t magic_number;
    uint16_t compress_type; // 压缩算法
    uint32_t uncompress_len; // 压缩前长度
    uint32_t compress_len; // 压缩后长度
    uint64_t reserved;
};

Message DayuDataFileMessage {
    required DayuDataFileType type = 1; // 区分文本、PB、或其它格式 (HTML, XML, JSON, ...)
    optional string proto_message_name = 2; // 如果是PB数据, 此PB数据的message名称。如果是文本, 则此字段可以不填。
    repeated string body = 3; // 真实的用户数据, repeated支持多条合在一个message里。
};
```

5 系统设计

5.1 整体方案描述

如前文中描述的分级过滤的方案, 在一个过滤分级内, 策略需要采用类MapReduce的模型计算统计特征, 这往往需要多模块的协作, 完成日志解析, 特征抽取, 统计计算以及策略过滤等步骤。除了统计过滤层级, 在点击过滤系统中存在较轻量级的过滤层级需求, 此级过滤只采用单条日志特征, 在单模块中就能完成完整的过滤流程。将此轻量级的过滤层级独立出来, 相对于统计过滤层级, 它无状态, 设计和部署也相对简单。综上所述, 我们需要提供两种过滤层级, 称之为简单过滤层级和统计过滤层级。系统设计方案如下图所示。两种过滤层级实现上采用相同的类组件。



统计过滤层级不仅能依赖单条日志特征也能够计算上下文统计特征完成过滤判定。采用和remora类似的统计特征计算方式。主要包含XExtractor、XAccumulator、TaskMerger、XFilter、LogMerger五种Worker。XExtractor读取原始日志，抽取特征并分片输出。多个XAccumulator实例并行工作，分别处理其对应的分片，计算特征。TaskMerger合并相同userkey的特征Task。XFilter将所有相同logid的特征值聚合在一起，形成过滤样本（logid: set），并调用策略组件完成过滤并输出每个策略过滤结果。最后LogMerger对所有策略结果进行综合判定，将过滤信息填写到原始日志后按需求拆分输出。完整的原始日志只会在首尾模块中会被读取。

对于检索日志过滤的需求，当前会在XFilter和LogMerger之间增加Joiner  点击保护功能，点

击过滤结果会当作一个策略结果join到其他策略过滤结果中。点击日志过滤系统部署中不需要增加此模块。

统一架构完成后，点击检索系统将采用同构的模块，分离的部署，点击系统的部署是一个简单过滤层级和两个统计过滤层级的串联。检索系统则采用单个统计过滤层级。

本章其他章节将分别介绍每个Worker的功能、实现要点以及输入输出。

5.2 XExtractor

XExtractor的功能是逐条从原始日志中抽取特征，对所有抽取的FeatrueValues进行分shard，排序，传递给XAccumulator进行特征统计。同时XExtractor也会将原始日志及joinkey传递给LogMerger。为保证FeatureValue和Log能正确拼接，XExtractor会分别对所有输出Task填写相同的userkey。XExtractor处理流程如下：

- 1. 读取日志文件，反序列化为LogRecord。
- 2. 根据策略配置抽取FeatureValue，并填写局部空间坐标，更新空间坐标累积量。
- 3. 抽取LogRecord joinkey。
- 4. 分片并排序FeatureValues。需要保证对于每个输入task，都生成预先配置好的PARTITION_NUMBER个特征Task，即便某个Partition中没有文件。
- 5. 写文件，分别创建XExtractor4XAccumulator和XExtractor4LogMerger task。

XExtractor多实例并行消费，采用QueryAndOwn的方式获取Task。XExtractor本地无状态，同构部署，增加工作实例即可水平扩展。

输入

	Group	Task	Data
原始日志	BundlerOutputGroup	DataTaskValue	SPLog(serialized_raw_log)

输出

	Group	Task	Data
特征	XExtractor4XAccumulator.X(0,n-1)	DataTaskValue	SPLog(CoordData+FeatureValue)
原始日志	XExtractor4LogMerger	DataTaskValue	SPLog(RawLogRecord)

```

message FeatureValue; // 定义见数据层接口描述

message CoordData {
    message CoordUnit {
        required int32 policy_index = 1;
        required int32 coord_delta = 2;
        optional int32 feature_id = 3;
    }
    repeated CoordUnit coord_data = 1;
}

message RawLogRecord {
    required uint64 search_id = 1;
    required string raw_log = 2;
    required string standard_log = 3;
    optional uint64 joinkey = 4;
    optional uint64 search_time_ms = 5;
    optional uint64 pv_time = 6;
}

```

5.3 XAccumulator

XAccumulator负责特征计算。作为过滤系统中唯一一个有状态的Worker，在设计中需要考虑更多异常恢复。

综合点击检索现有策略，从框架来看XAccumulator需要支持两种统计特征计算模型。

- 更新-获取特征 更新内存状态的同时获得更新后的特征值。包括基本特征和统计特征。
- 更新-延迟获取特征 主要解决时差策略的对日志时序敏感的需求。更新内存状态后，缓存Task一定时间间隔之后重新处理，并获取更准确特征值。

实现上可以沿用Remora-Filter的方案。两类计算模型不共享Shard Group和特征文件。第一类特征由Extractor分发到A1-An Group，第二类特征则分发到B1-Bm Group。XAccumulator同质部署，利用Token确定实际运行实例的角色。

状态持久化

XAccumulator使用HDFS上的Checkpoint来保存状态。实例获得token进入运行状态后，加载checkpoint获得状态。释放token退出运行状态或接收到指定信号时倒出checkpoint。

策略checkpoint保存着实例上一次退出时的策略状态，策略参数的修改可能导致checkpoint失效，策略自适应地发现配置的兼容性过于复杂，当前可以采用简单的解决方法：checkpoint存储策略版本号，实例启动后首先读取配置中的策略版本号，然后和checkpoint中的策略版本号比较，如果发现配置中的版本出现更新且策略不一致，则丢弃掉已有的checkpoint。

容错

XAccumulator内存维护统计状态。容错方案如"设计思路与折衷"章节中所述，根据不同的场景选择不同的容错方案。点击日志过滤部署采用每个shard三副本热备，检索日志过滤系统则采用每个shard单点。

输入

	Group	Task	Data
特征	XExtractor4XAccumulator.X(0,n-1)	DataTaskValue	SPLog(CoordData+FeatureValue)

输出

	Group	Task	Data
特征	XAccumulatorOutputGroup.X(0,n-1)	DataTaskValue	SPlog(FeatureValue)

5.4 TaskMerger

TaskMerger是轻量级模块，只处理Task。消费所有XAccumulator生成的特征Task，合并userkey相同的Task。TaskMerger能够在数据流经过XAccumulator后迅速减少Task个数，减少TM的压力。

Merge功能的实现依赖于TM提供的query_and_own_by_userkey批量query_and_own接口。TaskMerger每次处理时先QueryAndOwn第一个输入Group中的一个task。如果成功则并通过query_and_own_by_userkey接口尝试获取其余统计特征Group下userkey相同的Task。如果尝试失败则释放所有Task。合并后生成一个DataTaskValue。

TaskMerger多实例并行消费，采用QueryAndOwn的方式获取Task。TaskMerger本地无状态，同构部署，增加工作实例即可水平扩展。

输入

	Group	Task	Data
特征	FeatureOutputGroup.X	DataTaskValue	SPLog(FeatureValue)

输出

	Group	Task	Data
特征	TaskMergerOutputGroup	DataTaskValue	SPLog(FeatureValue)

5.5 XFilter

FeatureMergerAndFilter输入TaskMerger的输出，对每个Task中的所有特征文件按照logid汇聚，在汇聚之后中根据配置请求外部服务（特征中心）补全特征，得到特征集合，可以表示为logid:set(feature_value)。策略组件接受特征集合作为输入，输出此样本的作弊判定结果，用bool类型表示是否命中策略。同时支持在过滤结果消息中传递附属信息(q值)给下游，便于统计和实验。

XFilter多实例并行消费，采用QueryAndOwn的方式获取Task。XFilter本地无状态，同构部署，增加工作实例即可水平扩展。

输入

	Group	Task	Data
特征	TaskMergerOutputGroup	DataTaskValue	SPLog(FeatueValue)

输出

	Group	Task	Data
判定结果	XFilterOutputGroup	DataTaskValue	SPLog(PolicyResult)

message PolicyResult的定义见过滤层组件接口。

5.6 LogMerger

LogMerger负责消费XFilter生成的过滤结果和XExtractor输出的原始日志，将过滤结果和原始日志merge后拆分输出Normal和Spam日志。

Merge功能的实现和TaskMerger一样，依赖于TM提供的query_and_own_by_userkey批量query_and_own接口。LogMerger每次处理时先QueryAndOwn一个原始日志Group中的一个task。如果成功则并通过query_and_own_by_userkey接口尝试获取过滤结果Group下userkey相同的Task。如果尝试失败则释放所有Task。成功则消费两个Task，按照logid拼接过滤结果和原始日志，将过滤结果写入原始日志后

LogMerger多实例并行消费，采用QueryAndOwn的方式获取Task。LogMerger本地无状态，同构部署，增加工作实例即可水平扩展。

输入

	Group	Task	Data
原始日志	XExtractor4LogMerger	DataTaskValue	SPLog(RawLogRecord)
判定结果	XFilterOutputGroup	DataTaskValue	SPLog(PolicyResult)

输出

	Group	Task	Data
带有过滤 标记原始 日志	LogMergerOutputGroup. {A,B}	DataTaskValue	SPLog(serialized_raw_log)